

PROJECT ASSIGNMENT SPECIFICATION

Project Title

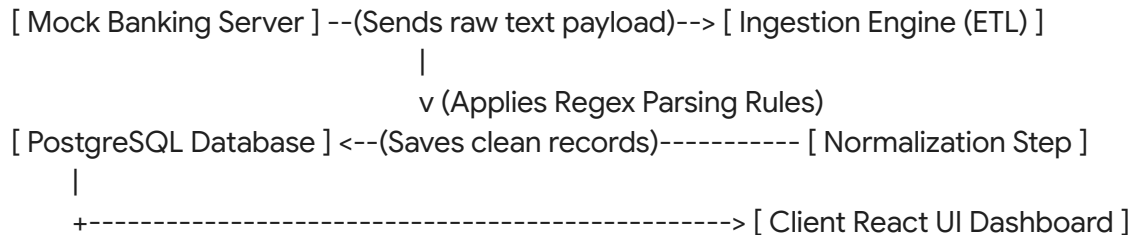
NexusFinance: Personal Finance Dashboard & Mock Open Banking Pipeline

Project Overview & Context

Modern financial applications (such as Mint, Copilot, or Monarch Money) rely heavily on retrieving user transaction histories from external financial institutions. In the real world, this is achieved using third-party APIs like Plaid or Yodlee.

In this internship project, you will build a complete, end-to-end simulation of this ecosystem. Instead of integrating with a paid, live Plaid environment, you will develop a **Mock Banking API (Sandbox)** that acts as your open-banking data provider. You will build a robust **ETL (Extract-Transform-Load) Pipeline** that pulls, cleans, and categorizes raw transaction payloads before storing them in a secure, relational database. Finally, you will design a **Client Dashboard (Frontend)** to visualize cash flow trends, manage credentials, and configure transaction parsing patterns.

Architectural Workflow



Technical Stack Recommendations

- **Backend:** Node.js (Express) OR Python (FastAPI/Flask)
- **Database:** PostgreSQL (highly recommended) or SQLite (for local development)
- **Frontend:** React with Tailwind CSS
- **State Management & Utilities:** Lucide React (Icons), Recharts (Visualizations)

Phase-by-Phase Core Requirements

Phase 1: Database & Mock Banking API

You must model and implement a relational database to support accounts and standardized transaction ledgers.

1. Relational Database Schemas (PostgreSQL / SQL)

You are expected to deploy the following schema models:

-- 1. User Accounts Table

```
CREATE TABLE accounts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  plaid_item_id VARCHAR(255) UNIQUE,  
  institution_name VARCHAR(100) NOT NULL,  
  account_name VARCHAR(100) NOT NULL,  
  account_type VARCHAR(50) NOT NULL CHECK (account_type IN ('Checking', 'Savings',  
'Credit Card', 'Investment')),  
  balance DECIMAL(12, 2) NOT NULL DEFAULT 0.00,  
  is_connected BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 2. Transactions Ledger Table

```
CREATE TABLE transactions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  account_id UUID REFERENCES accounts(id) ON DELETE CASCADE,  
  raw_merchant_string TEXT NOT NULL,  
  normalized_merchant_name VARCHAR(100) NOT NULL,  
  category VARCHAR(50) NOT NULL,  
  amount DECIMAL(10, 2) NOT NULL,  
  date_cleared DATE NOT NULL,  
  is_anomaly BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- 3. Dynamic Classification Rules Map Table

```
CREATE TABLE classification_rules (  
  id SERIAL PRIMARY KEY,  
  keyword VARCHAR(100) UNIQUE NOT NULL,  
  target_category VARCHAR(50) NOT NULL,  
  clean_merchant_name VARCHAR(100) NOT NULL  
);
```

2. The Mock Banking API Server

Implement mock HTTP endpoints that simulate an institution returning transactions when queried:

- **GET /api/sandbox/institutions:** Returns metadata about mock institutions (e.g., Apex

Bank, Horizon Union).

- **GET /api/sandbox/transactions:** Returns a randomized list of raw transaction objects.
 - *Payload structure:*

```
{
  "rawMerchant": "TST* BLUE BOTTLE COFFEE NY",
  "amount": 6.50,
  "clearedDate": "2026-06-20",
  "mockBank": "Apex Bank"
}
```

Phase 2: Ingestion & Normalization Engine (ETL)

This is the heart of the system. You must create a background worker or api controller to fetch and parse transactions.

1. Regex Classification Rule Processing

Raw bank statement data is highly unstructured (e.g., AMZN Mktp US*HJ291). Your code must read custom parser configurations from the `classification_rules` table and transform raw values:

- **Regex Check:** Match substring keywords (case-insensitive).
- **Mapping Example:** If raw string matches `/starbucks/i`, rewrite as Normalized Merchant: Starbucks and assign Category: Food & Dining.
- **Default Fallback:** If no rules match, store category as Uncategorized and status as Manual Review.

2. Audit & Anomaly Detection Filters

When transactions are ingested:

- Identify and flag anomalies (e.g., transactions where amount > \$100.00 or rapid duplicate velocities).
- Flag items as `is_anomaly = true` in the ledger so the user is immediately notified.

Phase 3: Client Dashboard (Frontend)

Build an intuitive, clear client dashboard interface.

- **KPI Metrics Cards:** Show overall combined cash balance, monthly expenditures, monthly gross income, and audit notifications.
- **Visual Charts (Recharts):**
 - **Donut Chart:** Expenditure volume segmented by categories (e.g., Food & Dining, Shopping, Utilities).
 - **Area/Line Chart:** Cash flow timeline showing Cumulative Expenses vs. Cumulative Incomes over time.
- **Dynamic Rules Manager:** An interface where users can add, delete, or test classification rules live, triggering updates in the mapping database.

- **Real-time Ledger Log Grid:** Sortable table to review and filter processed records by keyword and category.

Weekly Milestone & Deliverables Plan

- **Week 1: Foundations, DB Design & Mock Provider**
 - Set up local development repos, set up PostgreSQL database, and seed rules.
 - Write the Mock Bank API endpoints. Verify it returns random payloads successfully on Postman.
- **Week 2: Backend ETL Pipeline & Rule Parser**
 - Build the core processing endpoint (POST /api/ingest/sync).
 - Implement regex matching loops and anomaly audit algorithms. Write unit tests for your string-parsers.
- **Week 3: Web Dashboard UI Integration**
 - Develop the Dashboard UI with tables, charts, and configuration control tabs.
 - Connect Frontend forms to the Rule Config API so users can create categorization maps on-the-fly.
- **Week 4: Final Sandbox Flow Testing & Documentation**
 - Connect the entire chain: Link a Mock Bank account → trigger API fetch webhook → run clean engine → see dashboard charts update dynamically.
 - Write API Documentation (README.md) detailing setup steps. Provide a postman environment collection.

Stretch Goals (Bonus Credit)

- **Fuzzy Matching Search Engine:** Improve classification matching resilience by adding fuzzy string matching algorithms (such as Jaro-Winkler or Levenshtein Distance) for instances when exact regex checks miss.
- **Webhooks Handshake Simulation:** Build an async POST endpoint where your mock bank server triggers a TRANSACTION_SYNC event, initiating automated ingestion loops on your backend server.
- **Multi-Currency Support:** Support storing and converting currencies seamlessly inside your database models using fluctuating daily mock exchange rates.

Evaluation & Grading Rubric

Criteria	Outstanding (4)	Good (3)	Developing (2)	Needs Improvement (1)
Data Integrity & Schema	Proper foreign key	Clean schemas,	Lacks clear foreign keys;	Broken DB relationships,

	constraints, correct use of Decimal numeric types, transaction safety loops.	minor index optimization missed.	floats used for money values.	incorrect transaction logs.
Parsing Engine Accuracy	Regex functions flawlessly; implements fuzzy matching/robust fallbacks.	Regex maps reliably with clean categorization.	Partial regex support; fails to catch edge-case capitalizations.	System crashes or leaves all transactions uncategorized.
Dashboard UI/UX	Fluid responsive layouts, interactive Recharts, live data sync, clear empty-states.	Clear visual charts, clean dashboard, minor scaling issues.	Basic unstyled charts, functional data logs.	Broken frontend layouts, crashes on missing elements.
Documentation & Quality	Comprehensive README, API Swagger/Postman setups, modular structured code.	Readable codebase, setup steps provided in document.	Sparse comments, complex unorganized code folders.	No readme or setup instructions; code lacks modularity.